

Tipos de Comunicación en MPI

Alumno: Anthony Angel Briceño Quiroz
Curso: Computación Paralela y Distribuida

MPI ofrece 4 tipos de comunicación bloqueante para operaciones en envío (**send**), definidos según como se gestiona el buffer y la sincronización con el receptor.

- 1) **Standard (MPI_Send)**: Es el modo por defecto y más común que se usa en MPI, éste decide si el mensaje se almacena en un búfer temporal o lo envía directamente. Retorna cuando el búfer del remitente puede ser utilizado, lo cual puede ocurrir antes de que el receptor reciba el dato si hay espacio en el búfer.

```
1  #include <mpi.h>
2  #include <iostream>
3  #include <vector>
4
5  using namespace std;
6  #define N 12
7
8  int main(int argc, char ** argv){
9      int rank, size; //declaracion de variables usadas en MPI
10     MPI_Init(&argc, &argv);
11
12     /*Devuelve el identificador del proceso como tal en este caso rank*/
13     MPI_Comm_rank(MPI_COMM_WORLD,&rank);
14     /*Devuelve el tam o numero de procesos de un comunicador
15     en este caso el comunicador es MPI_COMM_WORLD*/
16     MPI_Comm_size(MPI_COMM_WORLD,&size);
17
18
19     int chunk = N / (size-1);
20     if (rank == 0){
21         vector<int>numeros;
22         for (int i = 1; i <= N; i++){
23             numeros.push_back(i);
24         }
25
26         for (int p = 1; p < size; p++){
27             MPI_Send(&numeros[(p-1)*chunk],chunk, MPI_INT,p,0,MPI_COMM_WORLD);
28         }
29
30         long long multiplicacionTotal = 1;
31         for (int p = 1; p < size; p++){
32             long long multiplicacionParcial = 1;
33             MPI_Recv(&multiplicacionParcial,1,MPI_LONG_LONG,p,1,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
34             multiplicacionTotal *= multiplicacionParcial;
35         }
36         cout << "Suma total: " << multiplicacionTotal << "\n";
37     } else {
38         vector<int> miParte(chunk);
39         MPI_Recv(miParte.data(),chunk,MPI_INT,0,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
40         long long multiplicacionParcial = 1;
41         for (int v: miParte){
42             multiplicacionParcial *= v;
43         }
44         MPI_Send(&multiplicacionParcial,1,MPI_LONG_LONG,0,1,MPI_COMM_WORLD);
45     }
46     MPI_Finalize();
47     return 0;
48 }
```

Bueno, básicamente lo que hace este código es multiplicar todos los elementos de un vector, o sea, del 1 hasta N, y esto lo hago usando comunicación estándar de MPI entre varios procesos. Entonces, el proceso con rank igual a 0, digámoslo así, es como el que coordina todo. Es el único que crea el vector y lo inicializa completo. Una vez que ya lo tiene listo, básicamente lo divide en partes iguales y le manda a cada uno de los otros procesos rank 1,2,3 y así su pedazo correspondiente, usando MPI_Send. Ahora, cada uno de esos otros procesos, o sea las otras minicomputadoras, recibe su parte del vector con MPI_Recv, y ahí multiplica solamente los elementos que le tocaron a él. Y esto pasa en paralelo, o sea, mientras un proceso está multiplicando su pedazo, los demás están haciendo exactamente lo mismo pero con su propia parte, al mismo tiempo. Cuando cada uno termina su cálculo, le manda ese resultado parcial de vuelta al rank cero, otra vez con MPI_Send. Y el rank cero recibe todos esos resultados con

MPI_Recv y los multiplica entre sí, y ahí ya se obtiene el resultado final, o sea el producto de todo el vector. Entonces, en resumen, sería como que un proceso reparte el trabajo, los demás procesan su parte en paralelo, y ese mismo proceso que repartió es el que junta y combina todo al final. Esa es la estructura básica de coordinador y trabajadores en MPI, usando el modo estándar de comunicación bloqueante.

- 2) Buffered (**MPI_Bsend**): El emisor copia inmediatamente los datos en un búfer asignado por el usuario o por el sistema. La llamada retorna tan pronto como la copia se completa, lo que permite separar al emisor del receptor. Este tipo de comunicación bloqueante es útil para evitar bloqueos, pero la desventaja es que consume mas memoria debido a la copia adicional que hace de los datos.

```
1  #include <mpi.h>
2  #include <iostream>
3  #include <vector>
4
5  using namespace std;
6
7  int main(int argc, char** argv){
8      int rank, size;
9      MPI_Init(&argc, &argv);
10     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11     MPI_Comm_size(MPI_COMM_WORLD, &size);
12
13     // MPI_Bsend necesita un buffer propio, separado del que maneja MPI internamente.
14     // Aquí reservamos ese espacio y se lo entregamos a MPI con MPI_Buffer_attach.
15     int tamanoBuffer = MPI_BSEND_OVERHEAD + sizeof(int);
16     char* miBuffer = new char[tamanoBuffer];
17     MPI_Buffer_attach(miBuffer, tamanoBuffer);
18
19     if (rank == 0){
20         vector<int> precios = {100, 250, 80, 500, 30, 999};
21
22         for (int p = 1; p < size; p++){
23             MPI_Bsend(&precios[p-1], 1, MPI_INT, p, 0, MPI_COMM_WORLD);
24         }
25
26         int precioConDescuentoRecibido;
27         for (int p = 1; p < size; p++){
28             MPI_Recv(&precioConDescuentoRecibido, 1, MPI_INT, p, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
29             cout << "Precio con descuento recibido del proceso " << p << ": " << precioConDescuentoRecibido << "\n";
30         }
31     } else {
32         int precioOriginal;
33         MPI_Recv(&precioOriginal, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
34
35         int precioConDescuento = precioOriginal - (precioOriginal * 20 / 100);
36
37         MPI_Bsend(&precioConDescuento, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);
38     }
39 }
40
41 // liberar el buffer usado por MPI_Bsend antes de finalizar
42 MPI_Buffer_detach(&miBuffer, &tamanoBuffer);
43 delete[] miBuffer;
44
45 MPI_Finalize();
46 return 0;
47 }
```

Bueno, este código lo que hace es repartir precios de productos a varios procesos usando el modo buffered de MPI, o sea MPI_Bsend. La diferencia con el modo estándar es que aquí yo mismo tengo que reservar un espacio de memoria, un buffer, y decirle a MPI 'usa este espacio para guardar lo que envíe', eso se hace con MPI_Buffer_attach al inicio. Entonces el rank cero tiene un vector de precios, y le manda a cada uno de los otros procesos un precio distinto usando Bsend. Como uso Bsend, ese envío retorna de inmediato, no me quedo esperando nada, porque el dato ya se copió a mi buffer propio. Cada proceso que recibe su precio le aplica un 20% de descuento y me lo devuelve, también con Bsend. Y al final, el rank cero recibe todos esos precios con descuento y los imprime. Y antes de terminar el programa, hay que liberar ese buffer que reservé al principio, con MPI_Buffer_detach, si no lo hago se queda memoria reservada sin usar.

- 3) Synchronous (**MPI_Ssend**): Este es el modo más seguro de todos, No retorna nada de la llamada si es que el **receptor** no ha iniciado con la recepción de los datos (**handshake completo**). Esto garantiza que el mensaje ha sido aceptado por el **receptor** antes de que el **emisor** continúe.

```
1  #include <mpi.h>
2  #include <iostream>
3  #include <vector>
4
5  using namespace std;
6
7  int main(int argc, char** argv){
8      int rank, size;
9      MPI_Init(&argc, &argv);
10     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11     MPI_Comm_size(MPI_COMM_WORLD, &size);
12
13     if (rank == 0){
14         vector<int> edades = {15, 25, 8, 40, 17, 60};
15
16         for (int p = 1; p < size; p++){
17             // MPI_Ssend no retorna hasta que el proceso destino ya haya
18             // empezado a ejecutar su MPI Recv correspondiente (rendezvous real).
19             MPI_Ssend(&edades[p-1], 1, MPI_INT, p, 0, MPI_COMM_WORLD);
20         }
21
22         int resultadoRecibido;
23         for (int p = 1; p < size; p++){
24             MPI_Recv(&resultadoRecibido, 1, MPI_INT, p, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
25             if (resultadoRecibido == 1){
26                 cout << "Proceso " << p << ": es mayor de edad\n";
27             } else {
28                 cout << "Proceso " << p << ": es menor de edad\n";
29             }
30         }
31
32     } else {
33         int edadRecibida;
34         MPI_Recv(&edadRecibida, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
35
36         int esMayorDeEdad = (edadRecibida >= 18) ? 1 : 0;
37
38         MPI_Ssend(&esMayorDeEdad, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);
39     }
40
41     MPI_Finalize();
42     return 0;
43 }
```

Este código valida si una serie de edades corresponden a mayores o menores de edad, pero usando el modo synchronous de MPI, o sea MPI_Ssend. La particularidad de Ssend es que no retorna hasta que el proceso que recibe ya empezó a ejecutar su Recv, es una especie de apretón de manos real entre los dos procesos, por eso se le dice rendezvous. El rank cero tiene un vector de edades, y le manda una edad distinta a cada uno de los demás procesos con Ssend. Cada proceso recibe su edad, verifica si es mayor o igual a 18, y me devuelve un uno si es mayor de edad o un cero si es menor, también usando Ssend. Y el rank cero recibe esas respuestas y las va imprimiendo, diciendo si cada proceso resultó mayor o menor de edad. La diferencia clave frente al modo estándar es que con Ssend tengo la garantía de que hay una sincronización real entre el que envía y el que recibe, no como en modo estándar donde MPI decide internamente si guarda el dato en un buffer temporal o espera. Eso lo hace más seguro para debug, porque si hay algún error de coordinación entre procesos, se nota enseguida porque el programa se queda esperando en vez de fallar silenciosamente.

- 4) Ready (**MPI_Rsend**): Solo se llega a iniciar si es que el **receptor** ya ha publicado una operación de recepción (**recv**). Retorna inmediatamente, asumiendo que el receptor está listo. Ofrece el menor **overhead** pero es difícil de depurar y tiende a fallar si el receptor no está preparado.

```
1  #include <mpi.h>
2  #include <iostream>
3  #include <vector>
4
5  using namespace std;
6
7  int main(int argc, char**argv){
8      int rank, size;
9      MPI_Init(&argc, &argv);
10     MPI_Comm_rank(MPI_COMM_WORLD,&rank);
11     MPI_Comm_size(MPI_COMM_WORLD,&size);
12
13     int temperaturaRecibida;
14     MPI_Request miSolicitud;
15
16     if (rank != 0){
17         MPI_Irecv(&temperaturaRecibida,1,MPI_INT,0,0,MPI_COMM_WORLD,&miSolicitud);
18     }
19
20     MPI_Barrier(MPI_COMM_WORLD);
21     if (rank == 0){
22         vector <int> temperaturas = {12,34,45,56,22,80};
23         int elementosVectorTemperaturas = temperaturas.size();
24         for (int p = 1; p < size;p++){
25             MPI_Rsend(&temperaturas[p-1],1,MPI_INT,p,0,MPI_COMM_WORLD);
26         }
27     }
28
29     if (rank != 0){
30         MPI_Wait(&miSolicitud,MPI_STATUS_IGNORE);
31
32         if (temperaturaRecibida < 10){
33             cout << "La tempetatura con valor " << temperaturaRecibida << "° es Fria\n";
34         }
35         if (temperaturaRecibida >=10 && temperaturaRecibida < 30){
36             cout << "La tempetatura con valor " << temperaturaRecibida << "° es Normal\n";
37         }
38         else{
39             cout << "La tempetatura con valor " << temperaturaRecibida << "° Caliente\n";
40         }
41     }
42
43     MPI_Finalize();
44     return 0;
45 }
```

Bueno, básicamente lo que hace este código es repartir temperaturas a varios procesos usando el modo ready de MPI, o sea **MPI_Rsend**, que es el modo más rápido pero también el más riesgoso, porque necesita que el receive del otro lado ya esté listo antes de que yo envíe, si no, falla. Entonces, para que eso funcione bien, primero cada proceso que no es el rank cero hace un **MPI_Irecv**, que es como abrir la puerta pero sin quedarme esperando ahí parado, o sea, dejo todo listo para recibir pero sigo avanzando en mi código. Después viene un **MPI_Barrier**, que es como un punto de encuentro obligatorio, todos los procesos tienen que llegar ahí antes de que cualquiera pueda seguir. Esto es clave, porque así me aseguro de que, cuando el rank cero vaya a enviar, todos los demás ya pasaron por su **Irecv**, o sea, ya tienen la puerta abierta. Entonces ahí sí, después de la barrera, el rank cero, que es el que tiene el vector con las temperaturas, le manda a cada uno de los otros procesos su temperatura correspondiente, usando **MPI_Rsend**. Y por último, cada proceso que recibió su temperatura hace un **MPI_Wait**, que ahí sí lo bloquea hasta que el dato realmente haya llegado. Y una vez que llega, clasifica esa temperatura, o sea, dice si es fría, normal o caliente, y lo imprime. Entonces en resumen, el truco de este código es que uso **Irecv** en vez de un **Recv** normal, porque un **Recv** normal me hubiera dejado bloqueado antes de llegar a la barrera, y eso hubiera generado un deadlock, como me pasó al principio. Con **Irecv** puedo pasar la barrera sin bloquearme, y ya después, con el **Wait**, espero tranquilo a que el dato llegue de verdad.

Por otro lado se habla también del uso de MPI_Isend y MPI_Irecv los cuales son de mucha ayuda, en un ejemplo anterior se uso MPI_Irecv.

```
1  #include <mpi.h>
2  #include <iostream>
3  #include <vector>
4  #include <unistd.h>
5
6  using namespace std;
7
8  int main(int argc, char** argv){
9      int rank, size;
10     MPI_Init(&argc, &argv);
11     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
12     MPI_Comm_size(MPI_COMM_WORLD, &size);
13
14     int edadRecibida;
15     MPI_Request solicitudRecepcion;
16
17     // todos los procesos que no son el rank 0 dejan lista su recepcion,
18     // sin bloquearse, usando MPI_Irecv
19     if (rank != 0){
20         MPI_Irecv(&edadRecibida, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &solicitudRecepcion);
21     }
22
23     if (rank == 0){
24         vector<int> edades = {15, 25, 8, 40, 17, 60};
25         vector<MPI_Request> solicitudesEnvio(size);
26
27         // el envio tampoco bloquea: MPI_Isend retorna de inmediato para cada proceso
28         for (int p = 1; p < size; p++){
29             MPI_Isend(&edades[p-1], 1, MPI_INT, p, 0, MPI_COMM_WORLD, &solicitudesEnvio[p]);
30         }
31
32         // como los envios no bloquean, antes de terminar hay que confirmar
33         // que cada uno realmente completo, usando MPI_Wait
34         for (int p = 1; p < size; p++){
35             MPI_Wait(&solicitudesEnvio[p], MPI_STATUS_IGNORE);
36         }
37         cout << "Rank 0: confirmado que todos los envios se completaron\n";
38
39     } else if (rank % 2 == 0){
40         // procesos con rank par: esperan el dato de forma bloqueante con MPI_Wait
41         MPI_Wait(&solicitudRecepcion, MPI_STATUS_IGNORE);
42         cout << "Proceso " << rank << " (uso Wait) recibio edad " << edadRecibida << "\n";
43
44     } else {
45         // procesos con rank impar: consultan repetidamente con MPI_Test
46         // si el dato ya llego, sin bloquearse, y mientras tanto pueden hacer otra cosa
47         int yaLlego = 0;
48         int intentos = 0;
49         while (!yaLlego){
50             MPI_Test(&solicitudRecepcion, &yaLlego, MPI_STATUS_IGNORE);
51             intentos++;
52             if (!yaLlego){
53                 usleep(1000); // simula trabajo util mientras se espera el dato
54             }
55         }
56         cout << "Proceso " << rank << " (uso Test, " << intentos
57             << " intentos) recibio edad " << edadRecibida << "\n";
58     }
59
60     MPI_Finalize();
61     return 0;
62 }
```

Este código muestra cómo funciona la comunicación no bloqueante en MPI, usando las cuatro funciones juntas: `Isend`, `Irecv`, `Wait` y `Test`. El rank cero tiene un vector de edades, y le manda una a cada uno de los demás procesos, pero en vez de usar `Send` normal, uso `MPI_Isend`, que retorna de inmediato sin esperar a que el envío realmente se complete. Como hago varios `Isend` seguidos, cada uno me da un `Request` distinto, entonces guardo todos esos `Requests` en un vector, para después, con un `MPI_Wait` por cada uno, confirmar que todos los envíos sí terminaron antes de que el rank cero siga. Del lado de los que reciben, todos hacen primero un `MPI_Irecv`, que tampoco bloquea, solo deja registrada la intención de recibir. Y ahí es donde separo el comportamiento en dos grupos, para mostrar la diferencia entre `Wait` y `Test`: Los procesos con rank par usan `MPI_Wait`, que los bloquea completamente hasta que el dato realmente llega, es la forma más simple de esperar. Los procesos con rank impar usan `MPI_Test` dentro de un bucle, que les permite preguntar '¿ya llegó mi dato?' sin bloquearse, y si todavía no llegó, pueden aprovechar ese tiempo haciendo otra cosa —en mi ejemplo simulo eso con una pequeña pausa— y vuelven a preguntar. Cuento cuántas veces preguntaron antes de que el dato llegara, para que se note esa diferencia. Entonces la relación entre las cuatro funciones es: `Isend` e `Irecv` inician una operación de comunicación sin bloquear el programa, y te devuelven un `Request`, que es como un ticket que identifica esa operación pendiente. `Wait` y `Test` son las dos formas de usar ese ticket para saber si la operación ya terminó: `Wait` se queda esperando hasta que sí, y `Test` solo pregunta una vez y sigue de largo si la respuesta es que no. Ninguna de las cuatro funciona sola, siempre van en pareja: un `Isend` o `Irecv` necesita, en algún momento, un `Wait` o un `Test` para confirmar que el dato de verdad se envió o llegó.